UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES

Sistemas Distribuidos

Clase 11 – Resiliencia distribuida

Departamento de Computación

Ingeniería Informática – Comisión A

Pilar – Segundo cuatrimestre 2026

Equipo docente

2026

1 Introducción

En un sistema distribuido la falla parcial no es una anomalía: es la condición normal de operación. Alguna dependencia está siempre más lenta de lo habitual, algún nodo se reinicia, alguna conexión se corta. La clase 10 mostró cómo *ver* esas fallas; esta clase trata de qué hace el sistema *mientras* ocurren. La pregunta de fondo no es cómo evitar toda falla —eso no es posible—, sino cómo impedir que una falla local se convierta en una falla global.

1.1 Falta, error y falla

Para hablar con precisión de la cadena que lleva de un defecto a una caída conviene distinguir tres términos.[1] Los ejemplos de cada uno salen del caso de una aerolínea.[2]

***Fault* (falta):** condición que crea un estado interno incorrecto en el software. Puede deberse a un defecto latente que se dispara o a una condición no controlada en una frontera con otro sistema. *Ejemplo:* tras una conmutación de la base de datos, cerrar una consulta lanza una excepción que el código no maneja, y la conexión a la base no se devuelve al pool. Nadie lo ve todavía, pero el pool pierde una conexión en cada llamada.

***Error*:** comportamiento visiblemente incorrecto. *Ejemplo:* los primeros llamantes del servicio de búsqueda de vuelos empiezan a recibir excepciones en lugar de resultados.

***Failure* (falla):** un sistema que no responde. *Ejemplo:* agotado el pool, el servicio de búsqueda de vuelos deja de responder, y con él los quioscos de *check-in* que lo llaman (las terminales de autoservicio de los aeropuertos en las que los pasajeros hacen el *check-in* sin pasar por el mostrador).

La falla depende de quién mira. Nygard agrega que "Failure is in the eye of the beholder": una computadora puede estar encendida y no responder ningún pedido.[3] Que algo "falló" no es una propiedad del servidor sino del observador. Para un monitoreo que solo verifica que el proceso esté vivo o que una página de estado responda, el servidor está sano; para el usuario que no recibe respuesta, el sistema falló. En el caso de la aerolínea ocurrió exactamente esto: el monitoreo del servicio de vuelos consultaba solo una página de estado y lo mostraba sano mientras todos sus hilos estaban bloqueados.[4] La consecuencia práctica es que la salud de un servicio debe medirse desde el punto de vista de quien lo usa (la clase 10 lo llamó medir desde el cliente).

La grieta. Una falta se convierte en error y un error provoca una falla; así avanza una *grieta* (*crack*). En cada eslabón la grieta puede acelerar, frenar o detenerse. El acoplamiento fuerte la acelera: cuantas más vías de propagación existen, más oportunidades hay de que una falta menor termine en una caída total. El conjunto formado por el disparador original, el modo en que la grieta se propaga y el daño resultante se denomina *modo de falla* (*failure mode*).[5]

1.2 Diseñar los modos de falla

Negar que las fallas son inevitables quita el poder de controlarlas. Si el diseño no define cómo falla el sistema, falla de la manera que emerja, que suele ser peligrosa. A las protecciones que se construyen

[1] Nygard (2018), cap. 3.
[2] La historia completa está en la sección 11, "Caso de estudio: la excepción que dejó en tierra a una aerolínea" (pág. 19).
[3] Nygard (2018), cap. 3.
[4] La historia completa está en la sección 11, "Caso de estudio: la excepción que dejó en tierra a una aerolínea" (pág. 19). Fuente: Nygard (2018), cap. 2.
[5] Nygard (2018), cap. 3.

a propósito para que el daño no avance se las llama *crackstoppers* (el término es de James Chiles): así como un automóvil tiene zonas de deformación que fallan primero para proteger a los pasajeros, el sistema puede decidir qué funciones son indispensables y construir modos de falla que mantengan las grietas lejos de ellas.[6]

La pregunta de esta clase puede formularse entonces con precisión: dado que una dependencia va a fallar, ¿en qué punto de la cadena se corta la propagación y a qué costo?

1.3 El ejemplo recurrente

A lo largo de estos apuntes se usa un sistema simple, un *generador de contraseñas*. Un servicio recibe pedidos de usuarios y, para armar cada contraseña, llama repetidamente a cuatro servicios más: uno que devuelve letras mayúsculas, otro letras minúsculas, otro dígitos y otro caracteres especiales. Hay además una dependencia que no figura en el diagrama de arquitectura: el servicio de minúsculas, internamente, consulta al de dígitos. Cada pedido hace varias llamadas a cada servicio hijo (un *fan-out*), de modo que cualquier política que se aplique por llamada se multiplica por pedido. Para que las cuentas sean simples se supone que cada pedido hace 5 llamadas a cada servicio hijo. El ejemplo es deliberadamente pequeño: sirve para razonar con números, no porque el problema sea pequeño.

2 Anatomía de una falla en cascada

2.1 Puntos de integración

Los puntos de integración son la principal fuente de fallas de los sistemas en producción: "the number-one killer of systems".[7] Cada conexión con otro sistema es una oportunidad de que el problema de otro se vuelva propio. La tendencia a descomponer en servicios más chicos y a integrar proveedores externos agrava la situación, porque multiplica la cantidad de puntos de integración.

Una *falla en cascada* (*cascading failure*) ocurre cuando una grieta en una capa provoca una grieta en la capa que la llama. Para que ocurra se necesita un mecanismo que transmita la falla de una capa a otra: la falla "salta el hueco" cuando el comportamiento del llamante se desencadena por la condición de falla del proveedor. Con frecuencia el mecanismo es un pool de recursos que se agota por una falla en una capa inferior, y una llamada a un punto de integración sin *timeout* es una forma segura de provocarla. Los servicios con muchos llamantes (alto *fan-in*) propagan sus problemas más ampliamente y merecen un escrutinio especial.[8]

2.2 Falla rápida y falla lenta

No todas las fallas de una dependencia son iguales para quien la llama. Una conexión rechazada vuelve en milisegundos; un paquete de confirmación que se pierde deja al hilo bloqueado durante minutos antes de que salte una excepción. El hilo bloqueado no puede atender otros pedidos y la capacidad total se reduce; si todos los hilos terminan bloqueados, para todo fin práctico el servidor está caído: "Clearly, a slow response is a lot worse than no response."[9]

La consecuencia de diseño es contraintuitiva: una dependencia lenta es más peligrosa que una dependencia caída, porque la caída libera rápido los recursos del llamante y la lentitud los retiene.

[6] Nygard (2018), cap. 3.
[7] Nygard (2018), cap. 4.
[8] Nygard (2018), cap. 4, *Cascading Failures*.
[9] Nygard (2018), cap. 4.

2.3 Hilos bloqueados y pools agotados

La mayoría de los servicios atiende cada pedido con un hilo tomado de un conjunto finito. Mientras el hilo espera la respuesta de una dependencia no hace nada útil, pero sigue ocupado. La mayoría de las caídas reales no son procesos que terminan, sino procesos vivos con todos sus hilos esperando un resultado que no llega: el sistema no "se cayó", "se colgó".[10]

Esto explica una observación que desorienta a quien opera: con el sistema saturado, la utilización de CPU es baja. Los hilos esperan entrada/salida, no calculan. Un panel de CPU en verde es compatible con un servicio completamente bloqueado, y por eso la saturación debe medirse sobre el recurso que realmente se agota (hilos ocupados contra hilos disponibles), no sobre el procesador.

2.4 Reacciones en cadena y capacidades desbalanceadas

Hay otros dos mecanismos de propagación que conviene conocer.[11]

Reacciones en cadena. En una granja horizontal de cinco servidores iguales detrás de un balanceador, cada nodo atiende el 20 % de la carga. Si uno falla, los cuatro restantes se reparten el total y cada uno pasa a atender el 25 %. El aumento absoluto es de 5 puntos porcentuales, pero la carga de *cada* servidor crece un 25 % ($25/20 = 1{,}25$). Si la causa de la primera caída era una condición de carga (una fuga de memoria, una condición de carrera), los sobrevivientes ahora están más cargados y son más propensos a fallar, y cada baja recarga a los demás: con tres servidores cada uno atiende el 33 %, con dos el 50 %. En el caso extremo de un clúster de dos nodos, la carga del sobreviviente se duplica. Como el defecto está en todos los servidores (son iguales), la única manera de detener la reacción es corregir el defecto.

Capacidades desbalanceadas. Un servicio de *front end* dispone de 3000 hilos de atención y el servicio de agendamiento que consulta tiene solamente 450. Mientras el porcentaje de pedidos que llegan al servicio chico sea pequeño no hay problema, pero una promoción que lleve a muchos más usuarios a ese camino lo desborda: el *front end* siempre puede abrumar al *back end* porque sus capacidades no están balanceadas. Igualar capacidades suele ser inviable económicamente, de modo que tanto llamantes como proveedores deben construirse para resistir una avalancha de pedidos.

2.5 La ley de Little aplicada al pool de hilos

La relación entre carga, concurrencia y tiempo de respuesta tiene una forma cuantitativa simple. La **ley de Little** establece que, en un sistema estable, el número promedio de elementos en el sistema es el producto de la tasa de llegada por el tiempo promedio que cada elemento permanece:[12]

$$L = \lambda \cdot W.$$

Una analogía ayuda a fijar la idea. En una peluquería llegan en promedio 2 clientes por hora ($\lambda = 2$) y cada uno permanece 1,5 horas ($W = 1{,}5$): en cualquier momento hay en promedio $L = 2 \times 1{,}5 = 3$ clientes adentro. Si el local tiene solo 2 sillas, no da abasto, aunque nada esté "roto"; y si cada atención se alarga, W y con él L crecen sin que llegue un solo cliente más.

[10] Nygard (2018), cap. 4, *Blocked Threads*.
[11] Nygard (2018), cap. 4, *Chain Reactions* y *Unbalanced Capacities*. El ejemplo de los servidores está adaptado: el original usa ocho.
[12] Little (1961).

Aplicada a un servicio con un pool de hilos, λ es la tasa de pedidos por segundo, W es el tiempo que un pedido ocupa un hilo y L es la cantidad de hilos ocupados *necesarios* para sostener esa tasa. Si L supera el tamaño del pool, el servicio no puede atender la demanda: los pedidos se acumulan en una cola.

Ejemplo. El generador de contraseñas recibe $\lambda = 5$ pedidos por segundo y atiende con un pool de 8 hilos. Con las dependencias sanas, un pedido tarda unas decenas de milisegundos y se necesitan $L = 5 \times 0{,}05 = 0{,}25$ hilos: el pool es holgado. Si ahora cada llamada a una de las dependencias tarda 2 s, como cada pedido hace 5 llamadas secuenciales a esa dependencia, el pedido ocupa un hilo unos $5 \times 2 = 10$ s y la concurrencia necesaria pasa a ser $L = 5 \times 10 = 50$. Con 8 hilos, el servicio satura: por mucho que se agregue capacidad de cómputo, la tasa sostenible es $8/10 = 0{,}8$ pedidos por segundo. Obsérvese que ninguna dependencia "se cayó": alcanzó con que una se pusiera lenta. Cambió W, no λ.

Tiempo en cola y tiempo de procesamiento. Cuando los hilos están todos ocupados, los pedidos nuevos esperan en una cola (la cola de escucha del socket y las colas del servidor web). La suma del tiempo en cola y del tiempo de procesamiento se llama *tiempo de residencia* (*residence time*). Un error frecuente es medir solo el tiempo de procesamiento: el servicio cree que todo anda bien mientras sus clientes se quejan de lentitud. Dado que la cola es serial y el procesamiento multihilo, a la larga el tiempo de cola domina.[13] Por eso la latencia que ve el cliente es mayor que la que registra el servidor, y la diferencia entre ambas es precisamente el tiempo de espera.

La ley de Little deja de describir bien al sistema cuando se alcanzan los límites y la longitud máxima de las colas.[14] Sirve para dimensionar y para predecir la saturación, no para modelar el comportamiento detallado una vez saturado.

Falta un ingrediente que vuelve todo esto mucho peor: cuando los llamantes reciben errores o *timeouts*, reintentan, y esa carga extra llega justo al componente saturado. Se analiza en la sección 5, junto con los reintentos.

3 Resiliencia y sus herramientas

En este curso se entiende por **resiliencia** la capacidad de un sistema de *seguir funcionando ante fallas parciales*, aunque sea de manera degradada: una dependencia lenta o caída debe afectar solo a la parte del sistema que la necesita, por el menor tiempo posible, y de una forma que el usuario y el operador puedan entender. No es lo mismo que la disponibilidad total (que nada falle) ni que la tolerancia a fallas por redundancia (tener una réplica que reemplace a la que cae): es la disciplina de *contener* la propagación.

Las herramientas de esta clase se pueden ubicar sobre la cadena de propagación vista en la sección anterior. Cada una corta un tramo distinto:

[13] Nygard (2018), cap. 9, *Demand Control*.
[14] Nygard (2018), cap. 9.

Herramienta	Qué corta	Sección
Timeout	La espera indefinida que convierte una dependencia lenta en hilos bloqueados.	4
Reintento	Las fallas transitorias (las recupera), a costa de más carga.	5
Circuit breaker	La insistencia contra una dependencia que ya se sabe enferma.	6
Degradación elegante (*fallback*)	El error que llega al usuario: responde algo útil en su lugar.	7
Bulkhead	Que una parte del sistema agote recursos que comparten las demás.	8
Backpressure y *load shedding*	La demanda mayor que la capacidad: frenar al productor o rechazar a tiempo.	9

Ninguna alcanza sola, y combinadas pueden interactuar mal; la sección 10 trata cómo se ordenan dentro de una llamada.

4 Timeouts

4.1 Para qué sirven

Un *timeout* es el mecanismo que permite dejar de esperar una respuesta cuando se estima que no llegará. Su necesidad viene de la naturaleza de las redes: todo sistema es hoy un sistema distribuido y toda red es falible, de modo que el código no puede esperar para siempre una respuesta que quizá nunca llegue —"Hope is not a design method"—. Un *timeout* bien ubicado aísla la falla: el problema de otro servicio no tiene por qué convertirse en el propio.[15]

4.2 Por defecto, muchas librerías esperan para siempre

El *timeout* no viene puesto: hay que configurarlo. Muchas librerías de cliente esperan indefinidamente si no se les indica un plazo. Dos ejemplos:

- En Python, la librería `requests` no corta nunca un pedido si no se le pasa el parámetro `timeout`: sin él, el código puede quedar colgado minutos o más.[16]
- En Java, `URLConnection` interpreta un *timeout* de 0 como espera infinita, tanto para conectar como para leer.[17]

Lo mismo vale para los recursos internos de un servicio: todo pool que bloquea al hilo que pide un recurso (una conexión a la base, por ejemplo) debe tener un plazo máximo de espera, y toda primitiva de sincronización debe usarse en su forma con plazo.[18] En el caso de la aerolínea, el pool de conexiones bloqueaba sin límite.[19]

[15] Nygard (2018), cap. 5, *Timeouts*.
[16] Documentación de `requests`, sección *Timeouts*: "By default, requests do not time out unless a timeout value is set explicitly."
[17] Documentación de Java, `java.net.URLConnection`, métodos `setConnectTimeout` y `setReadTimeout`.
[18] Nygard (2018), cap. 5.
[19] La historia completa está en la sección 11, "Caso de estudio: la excepción que dejó en tierra a una aerolínea" (pág. 19).

4.3 Timeout de conexión y timeout de lectura

Una llamada remota tiene al menos dos etapas con modos de falla distintos, y las librerías ofrecen un *timeout* para cada una:

- **Timeout de conexión** (*connect timeout*). Acota el establecimiento de la conexión. Si el destino está caído o inalcanzable, la conexión o es rechazada inmediatamente (falla rápida) o no recibe respuesta (falla lenta). Como establecer una conexión dentro de un mismo centro de datos es rápido, puede ser muy corto.
- **Timeout de lectura** (*read timeout*; en Java y en Apache HttpClient, *socket timeout*, por la opción `SO_TIMEOUT` del socket). Acota cuánto se espera a que lleguen datos una vez establecida la conexión. Debe reflejar cuánto tarda la dependencia en operación normal.[20]

Algunas librerías ofrecen un tercer plazo: el tiempo máximo para obtener una conexión del pool del propio cliente (en Apache HttpClient, *connection request timeout*).

Un detalle importante: el *timeout* de lectura mide el tiempo *entre* datos recibidos, no el total. En `requests`, por ejemplo, es el tiempo que el cliente espera entre bytes enviados por el servidor; en la gran mayoría de los casos coincide con la espera hasta el primer byte. Una respuesta que llega en pequeños fragmentos espaciados puede tardar mucho más que el *timeout* sin que este salte. Para acotar el tiempo total hace falta, además, un plazo global de la operación (*deadline*).

4.4 Cómo elegir el valor

Un *timeout* demasiado corto convierte la variabilidad normal en errores; demasiado largo no protege. El valor debe salir de la **distribución de latencia de la dependencia**, medida en condiciones normales: típicamente el percentil 99 de la línea de base más un margen que absorba la variabilidad. Se retoma así la lección de la clase 10: el promedio no describe a nadie, y un *timeout* basado en la media cortaría una fracción grande de las llamadas normales, mientras que uno basado en el máximo observado casi nunca actuaría. Debe medirse del lado del cliente: la latencia que importa para decidir cuánto esperar es la que ve el llamante, que incluye la red y el encolamiento y que puede diferir de lo que registra el servidor.

Como el percentil se estima a partir de histogramas con cubos de ancho fijo, es una aproximación; alcanza para calibrar, pero conviene dejar margen y no presentarlo como un valor exacto. El valor debe revisarse cuando cambia la dependencia o su carga.

4.5 Timeouts en una cadena de llamadas

Si el servicio A llama al servicio B y B llama al servicio C, ¿hace falta conocer toda la cadena para elegir el *timeout* de A hacia B? Para el valor de partida, no: el percentil 99 de B *medido desde A* ya incluye lo que B espera a C en condiciones normales. En el generador de contraseñas, la latencia del servicio de minúsculas medida desde el llamante ya contiene la consulta interna al servicio de dígitos; basta con medirla.

Lo que la línea de base no muestra es el **peor caso interno**. Si B tiene su propio *timeout* hacia C, y quizá reintentos, en una falla de C la llamada a B puede tardar mucho más que su percentil 99 normal: B espera su *timeout* completo, reintenta, vuelve a esperar. Si el *timeout* de A hacia B es menor que ese peor caso, A abandona mientras B sigue trabajando: A cuenta una falla aunque B quizá

[20] Apache HttpClient 4.5, `RequestConfig`: `getConnectTimeout` y `getSocketTimeout`; documentación de `requests`, sección *Timeouts*.

hubiera respondido (con un resultado o con su propio error controlado), y B consume recursos para una respuesta que nadie va a recibir.

El tiempo máximo que puede tardar una llamada lógica con reintentos es

$$T_{\max} \;=\; (1+r)\cdot t \;+\; \sum_{i=1}^{r} e_i,$$

donde t es el *timeout* por intento, r es la cantidad de reintentos y e_i son las esperas entre reintentos (sección 5). La regla es que el $T_{\max}$ de cada llamada interna tiene que caber dentro del *timeout* de quien la origina. Por eso, en cadenas largas, no alcanza con elegir cada valor por separado: hay que repartir un presupuesto de tiempo de extremo a extremo, y conocer la cadena importa para revisar que los *timeouts* internos y sus reintentos quepan dentro de los externos.

4.6 Timeouts y fail fast

El patrón *Timeouts* y el patrón *Fail Fast* son "two sides of the same coin".[21] Los *timeouts* se aplican sobre todo a los pedidos salientes y protegen de la falla de otro; *Fail Fast* se aplica a los pedidos entrantes y consiste en rechazar de inmediato lo que el sistema ya sabe que no podrá procesar, en vez de gastar recursos para descartar el resultado. Un pedido que se sabe de antemano que fallará (porque un recurso necesario no está disponible, porque el *circuit breaker* de una dependencia requerida está abierto, porque los parámetros son inválidos) debe fallar al principio, no a mitad del trabajo.

Una advertencia necesaria: fallar rápido no es lo mismo que funcionar. Un *timeout* bien puesto evita la saturación del llamante, pero sigue habiendo pedidos que fallan; recuperarlos es tarea de los mecanismos que siguen.

5 Reintentos

Los reintentos se presentaron en la clase 02 como la respuesta natural a la pérdida de un mensaje, junto con la condición de idempotencia. Esta sección muestra su otra cara: cuándo, en lugar de recuperar una falla, la amplifican.

5.1 Cuándo destruyen: la retroalimentación

Al aumentar la carga, los recursos se retienen más tiempo, lo que alarga los tiempos de respuesta de los pedidos nuevos. En algún momento el tiempo de respuesta supera el *timeout* del llamante, que abandona el pedido original y probablemente dispare un reintento —"exactly when it hurts the worst"—.[22] El reintento suma carga sobre el componente que ya no da abasto, lo que alarga más los tiempos, lo que produce más *timeouts* y más reintentos.

Un ejemplo numérico:[23] un *backend* puede atender 10 000 consultas por segundo y rechaza las que exceden ese límite. El *frontend* le envía 10 100: rechaza 100. Esas 100 se reintentan y se suman a las nuevas, de modo que el *backend* recibe 10 200 y rechaza 200; luego 300, y así sucesivamente. Cada vez menos pedidos tienen éxito en el primer intento, y una fracción creciente del trabajo del *backend* se dedica a pedidos que van a fallar. Si el *backend* cae por esa carga, la que atendía se reparte entre los que quedan y el efecto se propaga (una reacción en cadena). Bajar la carga al nivel previo puede no alcanzar para salir: mientras los reintentos sigan, mantienen al *backend* sobrecargado.

[21] Nygard (2018), cap. 5.
[22] Nygard (2018), cap. 9.
[23] Beyer et al. (2016), cap. 22, *Retries*.

El reintento, que en la clase 02 era la solución a una pérdida de mensaje, frente a una dependencia saturada es el combustible de la cascada.

5.2 Cuándo ayudan

Un reintento es útil frente a una falla *transitoria*: un paquete perdido, una conexión reiniciada. Frente a una falla persistente o a una sobrecarga, es destructivo. Dentro de un centro de datos, la falla suele deberse a algo que anda mal en el otro extremo y los problemas tienden a durar un tiempo, de modo que un reintento inmediato probablemente falle otra vez; mientras tanto, el cliente que reintenta mantiene sus recursos ocupados más tiempo y puede empujar su respuesta más allá del *timeout* de su propio llamante.[24]

Dos condiciones previas: solo se reintentan operaciones **idempotentes** (repetirlas no cambia el resultado), y solo se reintentan las fallas que tiene sentido reintentar (*timeouts*, errores de conexión, errores 5xx; no errores 4xx ni de validación, que fallarían igual).

5.3 La cuenta de la amplificación

Un pedido que depende de n llamadas independientes, cada una con probabilidad de falla p, falla si falla alguna:

$$P(\text{falla del pedido}) = 1 - (1-p)^n.$$

Con $p = 10\,\%$ y $n = 5$ llamadas por pedido,

$$1 - 0{,}9^5 = 1 - 0{,}59 \approx 41\,\%.$$

Un 10 % de fallas por llamada se convierte en cuatro de cada diez pedidos fallidos: el *fan-out* amplifica.

Con un reintento. Ahora la llamada solo falla si fallan *los dos* intentos, el original y el reintento. Si son independientes, eso ocurre con probabilidad $p \cdot p = p^2 = 0{,}1 \times 0{,}1 = 1\,\%$. El pedido, que sigue haciendo 5 llamadas, falla con probabilidad $1 - 0{,}99^5 \approx 4{,}9\,\%$. La protección es considerable.

El costo. Cada llamada pasa a significar en promedio $1 + p = 1{,}1$ intentos (el reintento solo ocurre cuando el primero falla), y con hasta dos reintentos, $1 + p + p^2 = 1{,}11$. Las 5 llamadas por pedido se convierten en unas $5{,}5$. Pero si la falla es una caída o una saturación, en lugar de una pérdida independiente, p se acerca a 1 y cada llamada pasa a significar $1 + r$ intentos: el reintento multiplica la carga justo sobre el componente que no puede soportarla.

Reintentos en varias capas. Si cada capa reintenta, los factores se multiplican. Con tres capas de tres intentos cada una (dos reintentos por capa) son hasta $3^3 = 27$ llamadas por cada pedido del usuario. Si las capas de *backend*, *frontend* y JavaScript emiten cada una 3 reintentos (4 intentos), una única acción del usuario puede generar $4^3 = 64$ intentos sobre la base de datos.[25] Conviene decidir en qué capa se reintenta y no reintentar en todas.

[24] Nygard (2018), cap. 5.
[25] Beyer et al. (2016), cap. 22.

5.4 Backoff exponencial

Para no reintentar de inmediato, se espera entre intentos. En el *backoff exponencial* la espera crece geométricamente con el número de reintento n ($n = 0, 1, 2, \dots$) hasta un tope:

$$d_n = \min\big(\text{tope},\ \text{base} \cdot m^n\big),$$

con m típicamente igual a 2.

Ejemplo con dos reintentos. Con base 100 ms, $m = 2$ y tope 1 s: el primer reintento espera $d_0 = 100$ ms y el segundo $d_1 = 200$ ms; si hubiera un tercero esperaría 400 ms, y a partir del quinto quedaría fijo en el tope de 1 s. Las esperas suman 300 ms. Con un *timeout* por intento de 500 ms, la llamada lógica puede tardar como máximo $T_{\max} = 3 \times 500 + 300 = 1800$ ms: ese es el número que debe caber en el *timeout* del llamante.

5.5 Jitter

Si todos los clientes que fallaron a la vez usan la misma espera d_n, también reintentan a la vez, y la ola de reintentos llega sincronizada. Es el fenómeno del *dogpile*: la carga transitoria que muchos clientes imponen al mismo tiempo, como cuando arrancan juntos varios servidores, cuando una tarea programada se ejecuta a la hora en punto en todas las máquinas o cuando un cambio de configuración se aplica a todos a la vez. Se mitiga dispersando los instantes con una variación aleatoria.[26]

El *jitter* agrega esa aleatoriedad a la espera. Dos de las variantes más usadas son:[27]

***Full jitter*:** se elige la espera uniformemente en el intervalo completo, $e_n \sim U(0,\, d_n)$.

***Equal jitter*:** se conserva la mitad del retardo y se aleatoriza la otra mitad, $e_n = d_n/2 + U(0,\, d_n/2)$.

Ejemplo. Tres clientes fallan al mismo tiempo y están en su segundo reintento, con espera nominal $d_1 = 200$ ms. Sin *jitter*, los tres reintentan exactamente a los 200 ms. Con *full jitter*, cada uno sortea un valor entre 0 y 200 ms; por ejemplo 30, 120 y 190 ms. Con *equal jitter*, cada uno sortea entre 100 y 200 ms; por ejemplo 110, 160 y 195 ms. *Full jitter* dispersa más, pero algún cliente puede reintentar casi de inmediato; *equal jitter* garantiza una espera mínima de la mitad del retardo, a cambio de dispersar menos.

El *backoff* con *jitter* es claramente mejor que el exponencial sin *jitter*, y debería considerarse un enfoque estándar para clientes remotos.[28] Su efecto solo es apreciable cuando hay muchos clientes que fallan a la vez: con pocos clientes no hay sincronización que romper.

5.6 Presupuesto de reintentos

Limitar los reintentos *por pedido* no alcanza para evitar la amplificación. Si cada pedido puede hacer hasta 3 intentos y toda la dependencia está sobrecargada, todos los pedidos los agotan: la carga crece hasta casi el triple justo cuando la dependencia menos la soporta. La idea del **presupuesto de reintentos** (*retry budget*) es que reintentar solo tiene sentido cuando las fallas son raras: si muchos pedidos están fallando, es señal de que el problema no es transitorio, y conviene dejar de reintentar.

Google combina dos límites:[29]

[26] Nygard (2018), cap. 4, *Dogpile*.
[27] Brooker (2015).
[28] Brooker (2015).
[29] Beyer et al. (2016), cap. 21, *Deciding to Retry*.

- **Por pedido:** como máximo 3 intentos. Si un pedido ya falló tres veces, el error se propaga al llamante.
- **Por cliente:** cada cliente lleva la cuenta de qué proporción de los pedidos que envía son reintentos, y solo reintenta mientras esa proporción esté por debajo del 10 %.

En el peor caso, con el límite por pedido solo, los pedidos crecen hasta algo menos del triple; agregando el límite por cliente, en el caso general el crecimiento queda en 1,1 veces. El segundo límite se ajusta solo: cuando las fallas son pocas, la proporción de reintentos es baja y se reintenta normalmente; cuando muchos pedidos fallan, la proporción sube, el cliente deja de reintentar y los errores se propagan, que es lo que corresponde cuando el problema no es transitorio. Una variante más simple es un tope por proceso, por ejemplo como máximo 60 reintentos por minuto; agotado el presupuesto, se falla sin reintentar.[30]

A estas reglas se suma una propia de los *circuit breakers* (sección siguiente): un rechazo del *breaker* nunca se reintenta, porque la decisión de no llamar ya incorporó la información de que la dependencia está mal.

5.7 Caso Pokémon GO

El lanzamiento de Pokémon GO de Niantic ilustra cómo los reintentos pueden convertir una sobrecarga en una cascada.[31] Antes del lanzamiento se había probado la carga hasta cinco veces la estimación más optimista; el ritmo real de pedidos por segundo fue casi cincuenta veces esa estimación. La sobrecarga volvió extremadamente lentos a los *backends* (en lugar de rechazar pedidos), y el balanceador de carga reintentaba los GET, sumando carga.

La aplicación, a su vez, reintentaba por cuenta de los usuarios con "a single immediate retry, followed by constant backoff". Durante la caída, los *backends* devolvían a veces gran cantidad de errores rápidos (por ejemplo, cuando se reiniciaba un *backend* compartido); esas respuestas sincronizaron los reintentos de los clientes y produjeron picos de pedidos de hasta veinte veces el máximo global anterior (el "*thundering herd*"). Entre las acciones de mitigación estuvieron aislar y ampliar el conjunto de balanceadores que servían al juego y limitar, con autorización de Niantic, el tráfico aceptado. Como cambio de fondo, Niantic incorporó *jitter* y *backoff* exponencial truncado a sus clientes.

La lección es doble: un reintento sin dispersión temporal convierte a muchos clientes independientes en una única ola, y la sobrecarga por reintentos puede alcanzar también a componentes ajenos al servicio que sufre el problema (en este caso, la capacidad de balanceo global).

6 Circuit breaker

6.1 Idea y estados

El nombre viene del dispositivo eléctrico: un componente que falla primero, cortando el circuito, para evitar que se destruya el sistema completo, y que luego puede restablecerse. En software, un *circuit breaker* envuelve una operación peligrosa y la *evita* cuando el sistema no está sano. Se diferencia así de los reintentos: existe para *prevenir* operaciones, no para reejecutarlas. Es una forma de degradar automáticamente la funcionalidad cuando el sistema está bajo estrés.[32]

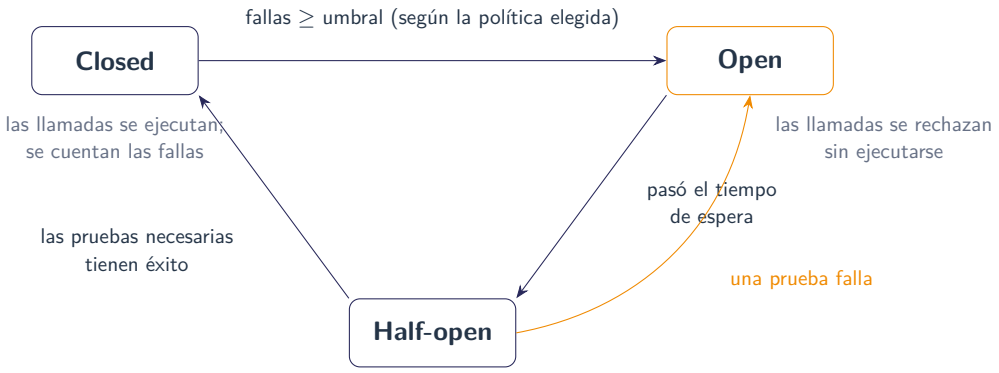
Se modela como una máquina de tres estados:

[30] Beyer et al. (2016), cap. 22.
[31] Bethea et al. (2018), cap. 11 del *SRE Workbook*.
[32] Nygard (2018), cap. 5, *Circuit Breaker*.

***Closed* (cerrado):** estado normal. Las llamadas se ejecutan y se registra su resultado. Si el conteo de fallas supera el umbral, el *breaker* "salta".

***Open* (abierto):** las llamadas fallan inmediatamente, sin intentar la operación real. Tras un tiempo de espera se pasa a *half-open*.

***Half-open* (semiabierto):** se deja pasar una llamada de prueba (o unas pocas). Si tiene éxito, el *breaker* se restablece y vuelve a *closed*; si falla, vuelve a *open* hasta que transcurra otro período de espera.



6.2 Qué cuenta como falla

El *breaker* solo debe contar los eventos que indican que la dependencia está enferma: *timeouts*, errores de conexión y respuestas 5xx. No debe contar errores del cliente (4xx) ni errores de validación: no dicen nada sobre la salud del proveedor, y contarlos abriría el *breaker* por culpa de pedidos incorrectos del propio llamante. Puede convenir además registrar por separado distintos tipos de falla, por ejemplo con un umbral más bajo para los *timeouts* que para las conexiones rechazadas. Cuando el *breaker* rechaza una llamada, conviene lanzar una excepción distinta de un *timeout* corriente, para que quien llama pueda reaccionar con información útil.[33]

La cooperación con los *timeouts* es un requisito, no una opción. Una llamada lenta que no tiene *timeout* no termina nunca, y el *breaker* solo registra un resultado cuando la llamada termina: sin *timeout* no ve la falla lenta, que es la más peligrosa. Por eso el *circuit breaker* se usa siempre junto con *timeouts*.

6.3 Cuándo abrir: fallas consecutivas contra tasa de fallas

La decisión central de configuración es cuándo se considera que hay "demasiadas fallas". Un simple contador que suma todas las fallas no sirve: hay una gran diferencia entre cinco fallas repartidas en cinco horas y cinco fallas en los últimos treinta segundos. Lo que importa es la *densidad* de fallas.[34] Existen dos familias de políticas.

Política de fallas consecutivas. El *breaker* se abre cuando se observan k fallas seguidas; cualquier éxito reinicia el conteo.

Ejemplo. Con $k = 3$, 10 s de espera en *open*, una llamada de prueba en *half-open* y un éxito para cerrar:

[33] Nygard (2018), cap. 5.
[34] Nygard (2018), cap. 5.

Llamada	Resultado	Fallas seguidas	Estado
1	éxito	0	*closed*
2	falla	1	*closed*
3	falla	2	*closed*
4	éxito	0 (se reinicia)	*closed*
5	falla	1	*closed*
6	falla	2	*closed*
7	falla	3	pasa a *open*

Durante los 10 s siguientes, todas las llamadas se rechazan sin ejecutarse. Pasado ese tiempo, el *breaker* pasa a *half-open* y deja pasar la próxima llamada como prueba. Si tiene éxito, vuelve a *closed* con el conteo en cero; si falla, vuelve a *open* por otros 10 s.

La política es simple, pero tiene una propiedad poco evidente. Si cada llamada falla con probabilidad p de forma independiente, el número esperado de llamadas necesarias hasta observar k fallas consecutivas es

$$E(k,p) = \frac{1-p^k}{(1-p)\,p^k}.$$

(Es el resultado clásico del tiempo de espera hasta una racha de longitud k en ensayos independientes de Bernoulli.) Los valores para dos umbrales y dos tasas de falla son:

	$p = 10\,\%$	$p = 50\,\%$
$k = 3$	≈ 1110 llamadas	14 llamadas
$k = 10$	$\approx 1{,}1 \times 10^{10}$ (en la práctica, nunca)	≈ 2046 llamadas

El dilema. Ningún valor de k sirve a la vez para tolerar una tasa de error baja pero normal (10 %) y para detectar rápido una dependencia realmente enferma (50 %):

- con $k = 3$ el *breaker* detecta rápido el 50 % (≈ 14 llamadas), pero también se dispara por azar con un 10 % de errores, cada ≈ 1110 llamadas;
- con $k = 10$ nunca se dispara con el 10 %, pero tarda ≈ 2046 llamadas en detectar el 50 %.

La causa es que las fallas consecutivas mezclan dos magnitudes distintas: la *tasa* de error y el *volumen* de tráfico. A igual tasa, cuanto más tráfico, más rachas ocurren por unidad de tiempo. A 100 llamadas por segundo, un *breaker* con $k = 3$ abre en promedio cada $1110/100 \approx 11$ s aun con un 10 % de errores; si permanece abierto 10 s cada vez, está abierto cerca del 47 % del tiempo y degrada el 100 % del tráfico hacia esa dependencia durante esos intervalos, por una tasa de error que el sistema toleraba perfectamente.

Política de tasa de fallas en ventana. La solución es medir la tasa directamente: el *breaker* mantiene una *ventana deslizante* con los resultados de las últimas N llamadas y se abre cuando la proporción de fallas alcanza un umbral θ, pero solo después de haber visto un mínimo m de llamadas, para no decidir con muestras insignificantes.

Ejemplo. Con una ventana de las últimas 10 llamadas, un mínimo de 5, un umbral del 50 %, 10 s de espera en *open* y una prueba en *half-open* (valores chicos para poder seguirlo a mano):

Llamada	Resultado	Fallas / llamadas	Tasa	Estado
1	falla	1 / 1	100 %	*closed* (menos del mínimo)
2	éxito	1 / 2	50 %	*closed* (menos del mínimo)
3	falla	2 / 3	67 %	*closed* (menos del mínimo)
4	falla	3 / 4	75 %	*closed* (menos del mínimo)
5	éxito	3 / 5	60 %	pasa a *open*

En las primeras cuatro llamadas la tasa supera el umbral, pero el *breaker* no decide porque no juntó el mínimo. En la quinta, con el mínimo alcanzado y una tasa del 60 %, abre. Pasados los 10 s, en *half-open* se deja pasar una prueba: si tiene éxito, vuelve a *closed* con la ventana vacía, y la tasa se vuelve a medir desde cero; si falla, vuelve a *open*. En cambio, con una falla cada diez llamadas, la tasa en la ventana es del 10 % y el *breaker* no abre nunca, cualquiera sea el volumen de tráfico.

Con valores más realistas, una ventana de 50 llamadas, un mínimo de 20 y un umbral del 50 %, la probabilidad de observar 25 o más fallas en 50 llamadas cuando la tasa real es del 10 % es del orden de 10^{-12} (distribución binomial): el *breaker* no abre. Con una tasa real del 50 % abre casi en cuanto se alcanza el mínimo, porque ya la primera evaluación con 20 llamadas tiene una probabilidad cercana al 59 % de ver 10 o más fallas. Detecta entonces una dependencia enferma en decenas de llamadas, independientemente del volumen total, y tolera la tasa baja. El compromiso se desplaza al tamaño de la ventana: una ventana chica detecta rápido pero es ruidosa (más falsos positivos cuando la tasa real se acerca al umbral); una ventana grande es estable pero reacciona más lento.

Una variante permite abrir también por latencia: un umbral de *llamadas lentas*. Aunque las llamadas no fallen formalmente, si una proporción grande supera una duración dada, la dependencia está degradada y la sobrecarga que genera en el llamante justifica cortarla.

6.4 Cuánto tiempo abierto y qué pasa en half-open

El tiempo que el *breaker* permanece abierto es un compromiso: si es muy corto, se prueba una dependencia todavía enferma una y otra vez y se acumulan pruebas fallidas; si es muy largo, se sigue degradando innecesariamente después de que la dependencia se recuperó. En *half-open* hay dos decisiones: cuántas llamadas de prueba se dejan pasar a la vez y cuántos éxitos hacen falta para cerrar. La formulación clásica deja pasar *la próxima llamada*, es decir, una sola.[35] Si la implementación deja pasar *todas* las llamadas durante *half-open*, cada hilo del llamante lanza una prueba simultánea contra una dependencia que quizá sigue mal, y se pierde el sentido de la prueba. Limitar las pruebas simultáneas es una decisión de configuración que conviene explicitar. Cerrar con un solo éxito es sensible a un éxito aislado; exigir varios éxitos reduce el riesgo de cerrar y volver a abrir en bucle.

6.5 Alcance

El *breaker* se construye con el alcance de un único proceso: el mismo estado afecta a todos los hilos del proceso, pero no se comparte entre procesos. Hay una pérdida de eficiencia, porque cada instancia del llamante descubre por su cuenta que el proveedor está caído, pero compartir el estado introduciría otra comunicación fuera de proceso, y con ella un nuevo modo de falla de la propia protección.[36] Corolario práctico: un *breaker* por dependencia y por proceso. Un único *breaker* para todas las dependencias haría que la falla de una desconecte a las demás.

[35] Nygard (2018), cap. 5.
[36] Nygard (2018), cap. 5.

6.6 Combinación con timeouts y reintentos

Dentro de una llamada lógica, cada intento (no cada llamada lógica) cuenta para el *breaker*: es la información sobre la salud de la dependencia. Si un intento fallido abre el *breaker*, el intento siguiente se rechaza y la secuencia de reintentos termina. Y el rechazo del *breaker* no se reintenta. Sin esa regla, el reintento anularía el efecto protector del *breaker*.

6.7 Observabilidad

Los cambios de estado de los *breakers* deben registrarse siempre y su estado actual debe poder consultarse. La frecuencia de cambios de estado es una métrica útil que conviene graficar en el tiempo, porque es un indicador adelantado de problemas en otras partes del sistema. Operaciones necesita además una forma de abrir o reiniciar el *breaker* manualmente. Es también un lugar cómodo para recoger métricas de volumen de llamadas y tiempos de respuesta.[37] Esto conecta con la clase 10: un *breaker* que cambia de estado sin que nadie lo vea es una decisión automática que se vuelve invisible.

6.8 Una protección mal implementada

El *breaker* es compartido por todos los hilos del proceso, de modo que su implementación tiene que ser **segura para hilos** (*thread-safe*): varios hilos pueden registrar éxitos y fallas y consultar el estado al mismo tiempo sin corromperlo (por ejemplo, sin perder conteos ni quedar en un estado inconsistente). Eso se logra protegiendo el estado compartido, típicamente con un *lock*.

Pero ser *thread-safe* no alcanza: importa *qué* protege el *lock*. Hay que evitar que el mecanismo termine serializando, sin quererlo, todas las llamadas a un sistema remoto.[38] Una implementación que toma el *lock* durante toda la ejecución de la llamada protegida es perfectamente *thread-safe*, y justamente por eso serializa todas las llamadas a la dependencia: mientras un hilo espera una respuesta lenta, los demás esperan el *lock*. La protección introduce la saturación que debía evitar.

Requisito doble para una librería de *circuit breaker*: que sea *thread-safe*, y que el *lock* cubra solo la lectura y la actualización del estado, nunca la llamada de red. Las librerías de resiliencia también se evalúan antes de adoptarlas: partir de una de código abierto es razonable, pero no exime de revisar cómo maneja la concurrencia.

7 Degradación elegante (graceful degradation)

7.1 El fallback como decisión de producto

Cuando una dependencia no está disponible (por un *timeout*, por un *breaker* abierto), el llamante debe hacer *algo* con el pedido. Hay varias estrategias de reemplazo (*fallback*): devolver la última respuesta buena o un valor en caché, una respuesta genérica en vez de una personalizada, o llamar a un servicio secundario. Cualquiera de ellas tiene impacto en el negocio, y por lo tanto es esencial involucrar a los interesados: ¿debe un sistema de comercio aceptar una orden si no puede confirmar la disponibilidad de los productos? ¿y si no puede verificar la tarjeta o la dirección? La decisión es de producto, no técnica; hablar del *circuit breaker* suele ser una forma más efectiva de abrir la conversación que pedir un documento de requisitos.[39]

[37] Nygard (2018), cap. 5.
[38] Nygard (2018), cap. 5: "Be sure to avoid accidentally single-threading all calls to a remote system!"
[39] Nygard (2018), cap. 5.

7.2 Condiciones de una buena degradación

- **Explícita para quien recibe la respuesta.** Si la respuesta es degradada, el receptor debe poder saberlo (por ejemplo, con un campo que lista qué partes faltan). Una respuesta degradada presentada como completa transforma una falla visible en un error silencioso.
- **Definida de antemano**, junto con los interesados, y no improvisada en la excepción.
- **Más barata que el camino normal**: una degradación que consume tantos recursos como la operación completa no contiene nada.

7.3 Cuándo no es aceptable degradar

Algunas operaciones no admiten una versión parcial. En el generador de contraseñas, omitir los caracteres especiales cuando ese servicio no responde produce una contraseña que puede violar la política de contraseñas del usuario: "degradar" pasa a ser "responder algo incorrecto". Lo mismo ocurre con un cobro cuyo servicio de verificación no responde. En esos casos la decisión correcta puede ser fallar de forma explícita (y rápida), o diferir la operación, en vez de simular un éxito.

7.4 Un breaker sin fallback

Un *breaker* sin estrategia de reemplazo falla rápido, pero sigue fallando: el llamante libera recursos y evita saturarse, pero los usuarios siguen recibiendo errores. El *circuit breaker* protege al sistema; solo el *fallback* (o una decisión de producto equivalente) protege a la experiencia del usuario. Por eso son mecanismos complementarios.

8 Bulkheads

Un barco se divide en *compartimentos estancos*: si el casco se perfora en un punto, el agua inunda ese compartimento pero no pasa a los demás, y el barco no se hunde. Los *bulkheads* son los mamparos que separan esos compartimentos, y dan nombre al patrón. Aplicado a un sistema, se trata de partir los recursos para que la falla de una parte no destruya el resto: "salvar parte del barco".[40]

8.1 Formas

Pueden particionarse recursos a distintos niveles de granularidad:[41]

- **Hilos dentro de un proceso:** un pool de hilos (o un tope de concurrencia) por dependencia, de modo que una dependencia lenta agote solamente su porción. Un caso particular útil es reservar un pool de hilos para tareas administrativas, para poder diagnosticar o apagar el servicio aun si todos los hilos de atención están colgados.
- **Procesadores de un servidor,** por ejemplo fijando un proceso a ciertos núcleos para que un proceso descontrolado no consuma todos los ciclos del equipo.
- **Instancias o clústeres dedicados:** capacidad reservada para clientes críticos, separada de la que usan los no críticos. Por ejemplo, un sistema de emisión de pasajes con servidores dedicados al *check-in*, que no se verían afectados si los servidores compartidos se saturan con consultas de estado de vuelo.
- **Zonas y regiones** en la nube.

[40] Nygard (2018), cap. 5, *Bulkheads*.
[41] Nygard (2018), cap. 5.

8.2 Costo

El *bulkhead* tiene un precio: capacidad ociosa. Una partición reservada para un cliente crítico está subutilizada la mayor parte del tiempo, y particionar demasiado fino desperdicia recursos. No hay una regla única: hay que evaluar el impacto de negocio de cada pérdida de capacidad y buscar los límites naturales donde la partición sea técnicamente factible y económicamente conveniente, alineados con los llamantes, con las funciones o con la topología.[42]

8.3 Un caso

En el caso de la aerolínea, el servidor del servicio de vuelos usaba pools de hilos separados para las llamadas remotas y para las peticiones HTTP. Por eso podía seguir respondiendo a la aplicación de monitoreo aun en medio de la caída, y el diagnóstico posterior fue posible.[43] Esa partición, sin embargo, no protegió a los demás sistemas: el recurso que se agotó (el pool de conexiones a la base) estaba dentro de la partición que atendía los pedidos de los clientes. Partir los servidores del servicio de vuelos en grupos dedicados a distintos clientes habría permitido seguir atendiendo el *check-in* de pasajeros aunque otros consumidores no pudieran consultar vuelos.[44]

9 Backpressure y load shedding

9.1 Las colas

"Every performance problem starts with a queue backing up somewhere."[45] Puede ser la cola de escucha de un socket, la cola de ejecución del sistema operativo o la cola de E/S de una base de datos. Si una cola no tiene límite, puede consumir toda la memoria disponible y, según la ley de Little, a medida que su longitud crece también lo hace el tiempo que tarda un trabajo en atravesarla: cuando la longitud tiende a infinito, el tiempo de respuesta también. Por eso no deben existir colas no acotadas. Con una cola acotada llena y un productor que intenta agregar un elemento más, las opciones son limitadas:

- aparentar aceptarlo y descartarlo;
- aceptarlo y descartar otro elemento de la cola (por ejemplo, el más viejo, si el valor de los datos decae rápidamente con la edad);
- rechazarlo;
- bloquear al productor hasta que haya lugar.

9.2 Backpressure

Bloquear al productor es una forma de control de flujo: la cola aplica presión hacia atrás (*back pressure*), que idealmente se propaga hasta el cliente final, cuya velocidad se reduce hasta que la cola libera espacio. TCP lo hace con la ventana de recepción. Funciona mejor con programación asincrónica y solo ayuda a administrar la carga cuando el conjunto de consumidores es finito; si el origen es muy diverso, no hay efecto sistémico. Hay que distinguir además una presión por una condición temporal de una presión porque un consumidor está roto.[46]

[42] Nygard (2018), cap. 5.
[43] La historia completa está en la sección 11, "Caso de estudio: la excepción que dejó en tierra a una aerolínea" (pág. 19). Fuente: Nygard (2018), cap. 2.
[44] Nygard (2018), cap. 5.
[45] Nygard (2018), cap. 5, *Create Back Pressure*.
[46] Nygard (2018), cap. 5.

9.3 Load shedding

Un servicio no controla su demanda: el mundo siempre puede generar más carga de la que se puede atender, y no se la puede superar solo escalando. Ante carga alta, lo mejor que se puede hacer es rechazar el trabajo que no se podrá completar a tiempo: el *load shedding* (descarte de carga), la forma más importante de controlar la demanda entrante.[47]

Ejemplo. Un servicio que procesa video en vivo (por ejemplo, el que recodifica una transmisión para distintos dispositivos) recibe más cuadros de los que puede procesar. Si los encola, cada cuadro espera más que el anterior y termina saliendo cuando la escena ya pasó: un cuadro que llega tarde no vale nada para el espectador, y procesarlo quita capacidad a los que todavía podrían llegar a tiempo. Lo correcto es descartar los cuadros que ya no van a llegar a tiempo y dedicar la capacidad a los que sí. El ejemplo es de la cátedra; la idea general es que, cuando el valor de un trabajo decae con la edad, descartar lo viejo es mejor que encolarlo.

Algunos principios:[48]

- un rechazo rápido es mejor que un *timeout* lento: el servidor sobrecargado debe fallar temprano y con poco costo (*fail early and cheaply*);
- conviene rechazar lo antes posible, antes de haber ocupado recursos en varias capas; los balanceadores cercanos al borde son el lugar ideal;
- "carga demasiado alta" puede definirse como el momento en que los pedidos tardan más que el acuerdo de nivel de servicio, o con un semáforo que limite los pedidos simultáneos;
- con un balanceador, cada instancia puede responder 503 en su *health check* para indicarle que se aparte por un tiempo, y el balanceador responde 503 al cliente cuando todas las instancias están indisponibles.

Backpressure desacelera al productor; *load shedding* rechaza trabajo que ya no puede procesarse a tiempo. Dentro de los límites de un sistema o de una empresa, con productores conocidos, conviene la *backpressure* y el *load shedding* queda como medida secundaria; en el borde, expuesto a una demanda efectivamente ilimitada, el *load shedding* es imprescindible.

9.4 Caso Dressy: dos mecanismos sanos que interactúan mal

El *SRE Workbook* presenta un caso ilustrativo (la empresa Dressy es explícitamente ficticia) de un servicio de venta de ropa desplegado en tres regiones.[49] La región A recibió ligeramente más tráfico que las otras y su CPU alcanzó antes el umbral configurado para el *load shedding*: cada servidor empezó a rechazar el 10 % de los pedidos, luego el 20 %, luego el 50 %. El balanceador de carga conocía la utilización de CPU de los contenedores y la usaba para estimar el costo por pedido; como la CPU se mantenía constante mientras los pedidos rechazados bajaban el costo aparente, concluyó que en A cada pedido costaba diez veces menos, es decir, que A era mucho más eficiente. A atendía 240 pedidos por segundo al 80 % de CPU (el tope de rechazo) mientras B y C atendían solamente 120; el balanceador siguió enviando más tráfico a A, hasta que A recibió 400 y B y C cayeron a 40.

En palabras del capítulo, "the load balancer didn't know that the 'efficient' requests were errors": los dos sistemas fueron agregados y habilitados por separado, probablemente por personas diferentes, y nadie los examinó como un único sistema de manejo de carga. La corrección propuesta es contar cada

[47] Nygard (2018), cap. 5, *Shed Load*, y cap. 9, *Demand Control*.
[48] Nygard (2018), caps. 5 y 9; Beyer et al. (2016), cap. 22.
[49] Bethea et al. (2018), cap. 11 del *SRE Workbook*.

pedido rechazado como 120 % de utilización de CPU (cualquier valor por encima de 100 sirve), con lo cual A pasa a parecer sobrecargada y el tráfico se reparte. La lección es general: cada mecanismo de control de carga debe configurarse y evaluarse junto con los demás, no por separado.

10 Combinar mecanismos

10.1 El orden dentro de una llamada

Los mecanismos de las secciones anteriores no son alternativas: se combinan en una misma llamada, y el orden en que se aplican importa. Un orden razonable, de afuera hacia adentro, es el siguiente.

1. **Bulkhead:** antes de gastar nada, ver si hay lugar en la partición reservada a esa dependencia; si no lo hay, rechazar de inmediato.
2. **Circuit breaker:** si la dependencia está marcada como enferma, rechazar sin llamarla. Va antes de la llamada y antes de los reintentos, porque es la decisión de no intentar.
3. **Timeout:** el intento real, con plazo acotado.
4. **Reintentos:** si el intento falló por una causa reintentable, esperar con *backoff* y *jitter* y volver a pasar por el *breaker*, de modo que cada intento cuente y que la secuencia se interrumpa si el *breaker* abrió.
5. **Fallback:** si todo falló o se rechazó, aplicar la degradación decidida de antemano.

La razón del orden es que cada mecanismo ahorra recursos al siguiente: el *bulkhead* y el *breaker* evitan intentos inútiles; el *timeout* acota los que se hacen; los reintentos recuperan lo transitorio sin saltarse las protecciones; el *fallback* decide qué hacer con lo que no se pudo recuperar.

10.2 Interacciones peligrosas

Tener cada mecanismo bien configurado por separado no garantiza que funcionen bien juntos. Ejemplos ya vistos: reintentos en varias capas que se multiplican y *load shedding* que engaña al balanceador (caso Dressy). Otro ejemplo, derivado de lo anterior: reintentos que ocupan el pool que un *bulkhead* intenta proteger. Al adoptar una nueva herramienta de manejo de carga hay que examinar con cuidado cómo interactúa con las demás; el manejo de tráfico funciona mejor cuando se lo piensa como un todo. En el caso de Pokémon GO, por ejemplo, una ola de reintentos sincronizados se combinó con balanceadores que esperaban a *backends* que no respondían.[50]

10.3 Más patrones no es mejor

Un sistema con más patrones no es superior a uno con menos: "Count of patterns applied" nunca es una buena métrica de calidad.[51] Lo que se busca es una mentalidad orientada a la recuperación: esperar que haya fallas y aplicar los patrones con criterio para reducir el daño de cada una. Cada mecanismo agrega configuración, estados y modos de falla propios; se justifica por la amenaza concreta que contiene, y su efecto debe verificarse (con pruebas de carga y de falla, no solo con revisión de código).

11 Caso de estudio: la excepción que dejó en tierra a una aerolínea

[50] Bethea et al. (2018), cap. 11 del *SRE Workbook*.
[51] Nygard (2018), cap. 5.

Este caso real, relatado por quien hizo el análisis posterior, recorre la cadena completa de una falla en cascada y permite ubicar en cada eslabón el mecanismo que la habría cortado.[52]

El sistema. Una aerolínea estaba migrando a una arquitectura orientada a servicios. Un servicio central, al que el relato llama CF (*core facilities*), resolvía la búsqueda de vuelos: dada una fecha, una ciudad, un aeropuerto o un número de vuelo, devolvía la lista de vuelos. Lo usaban los quioscos de *check-in* de los aeropuertos (las terminales de autoservicio en las que los pasajeros hacen el *check-in* por su cuenta), el sistema de respuesta telefónica automática y las aplicaciones que alimentaban a los sitios de venta de pasajes. CF se había diseñado para alta disponibilidad: un clúster de servidores de aplicación detrás de balanceadores redundantes, una base de datos Oracle en un clúster con conmutación automática, hardware redundante en todos los niveles y hasta un segundo centro de datos. No tenía ninguno de los puntos únicos de falla habituales. Sus clientes lo invocaban mediante llamadas remotas Java (EJB sobre RMI[53]).

La noche del cambio. Un jueves, cerca de las 23 h (hora del Pacífico), el equipo de operaciones ejecutó una conmutación manual de la base de datos de CF de un servidor al otro para hacer mantenimiento de rutina; era un procedimiento que habían hecho decenas de veces. El monitoreo mostró a las aplicaciones disponibles durante todo el cambio, y cerca de las 0:30 se lo dio por terminado con éxito.

La caída. Alrededor de las 2:30, todos los quioscos de *check-in* del país dejaron de atender al mismo tiempo; minutos después cayó también la respuesta telefónica. En la costa este eran las 5:30, hora pico de *check-in* de los vuelos de la mañana. El único elemento que ambos sistemas tenían en común era CF, que además había tenido la conmutación de la base tres horas antes. Sin embargo, el monitoreo de CF no mostraba problemas: solo consultaba una página de estado, que seguía respondiendo. Para restablecer el servicio se reiniciaron los servidores de CF, con lo que se recuperó la respuesta telefónica; los quioscos recién se recuperaron cuando se reiniciaron también sus propios servidores de aplicación. El incidente duró algo más de tres horas, pero sus consecuencias, mucho más: la aerolínea no tenía suficiente personal para hacer el *check-in* con los sistemas anteriores, tuvo que convocar empleados fuera de turno y recién cerca de las 15 h terminó de atender el atraso, con muchos vuelos demorados.

La causa. El análisis posterior, con los volcados de hilos tomados durante la caída, mostró lo siguiente. En los servidores de los quioscos, los 40 hilos que atendían pedidos estaban todos bloqueados esperando la respuesta de una llamada a CF. En CF, todos los hilos que atendían esas llamadas estaban bloqueados en la misma línea: esperando obtener una conexión del pool de conexiones a la base. El código de la búsqueda de vuelos cerraba la consulta y la conexión dentro de un bloque `finally`, pero cerrar la consulta podía lanzar una excepción (la especificación de JDBC lo permite), y eso es exactamente lo que ocurría con las conexiones abiertas antes de la conmutación, que habían quedado rotas. La excepción impedía llegar a cerrar la conexión, de modo que cada llamada dejaba una conexión sin devolver. Tras unas cuarenta llamadas el pool se agotó, y desde entonces toda llamada quedó bloqueada esperando una conexión que nunca iba a liberarse. Una sola excepción no manejada paralizó a una aerolínea.

La cadena y sus puntos de corte. La tabla recorre la cadena eslabón por eslabón y marca qué mecanismo de esta clase la habría cortado.

[52] Nygard (2018), cap. 2 (el relato) y cap. 3, *Stopping Crack Propagation* (los puntos de corte).

[53] RMI (*Remote Method Invocation*) es el mecanismo de RPC de Java: permite invocar un método de un objeto que vive en otra máquina como si fuera local (ver clase 02 sobre RPC). Sus llamadas, por defecto, no tienen *timeout*: el llamante puede quedar esperando indefinidamente. Nygard (2018), caps. 2 y 3.

Eslabón de la cadena	Mecanismo que la corta	Por qué
Excepción al cerrar la consulta; la conexión no se devuelve	Revisión y pruebas de falla (no es un patrón de resiliencia)	Es la falta original; no todos los defectos se pueden eliminar, de modo que hay que sobrevivirlos.
Pool agotado; hilos bloqueados esperando una conexión	*Timeout* en el pool (bloquear un tiempo limitado)	El hilo se libera y el error queda local.
Llamadas remotas sin *timeout*	*Timeout* en el cliente	Evita que la lentitud de CF se vuelva bloqueo de los quioscos.
Cada cliente bloquea hilos esperando a CF	*Circuit breaker* en el cliente	Tras unas fallas deja de llamar y falla rápido.
CF compartido por todos los clientes	*Bulkhead* (grupos de servidores, capacidad reservada)	Un problema en un grupo no tumba a todos los usuarios de CF.
Quioscos y respuesta telefónica caídos	*Fallback* o degradación	Decidir con el negocio qué hacer sin la búsqueda de vuelos.
Monitoreo que solo miraba una página de estado	Medir desde el cliente (clase 10)	La falla habría sido visible antes.

El análisis original enumera varios de esos puntos de corte: el pool podría haberse configurado para bloquear solo por un tiempo limitado; el cliente de RMI podría haber instalado un *timeout* en los sockets; los servidores de CF podrían haberse separado en grupos de servicio; y arquitecturas menos acopladas (colas de pedido y respuesta) habrían amortiguado el error en lugar de amplificarlo.[54] La conclusión es la que organiza esta clase: no es realista esperar que se eliminen todos los defectos, de modo que hay que sobrevivirlos; y la pregunta útil es cómo evitar que un defecto en un sistema afecte a todos los demás.

12 Modelo del curso

Expresamos el problema de la resiliencia mediante el framework del curso:

$$D = (P, F, G, S, T)$$

- P **(Problema):** Impedir que la falla o la lentitud de una dependencia se propague al resto del sistema.
- F **(Factores/Restricciones):** Fallas parciales, dependencias lentas, pools finitos de hilos, *fan-out*, reintentos que amplifican la carga, tráfico que no se controla.
- G **(Garantías/Objetivos):** Contención del daño, latencia acotada, recuperación automática y degradación explícita en vez de caída total.
- S **(Solución):** *Timeouts* calibrados con percentiles, reintentos con *backoff* y *jitter* acotados por un presupuesto, *circuit breaker* por dependencia con política de tasa de fallas, *fallback* decidido con el negocio, *bulkheads*, *backpressure* y *load shedding*.
- T **(Trade-offs/Costos):** Capacidad ociosa (*bulkheads*), degradación de la respuesta, complejidad de configuración, falsos positivos y falsos negativos del *breaker*, interacciones entre mecanismos.

[54] Nygard (2018), cap. 3.

13 Conclusión

La resiliencia distribuida no consiste en evitar las fallas ni en agregar reintentos sin más: consiste en decidir de antemano cómo falla el sistema. Cada mecanismo corta un tramo de la cadena de propagación: los *timeouts* acotan la espera, los reintentos recuperan lo transitorio, el *breaker* deja de insistir contra una dependencia enferma, el *fallback* decide qué recibe el usuario, los *bulkheads* limitan el alcance del daño y el *load shedding* protege al servicio de la demanda que no puede atender. Ninguno es gratuito, y en combinación pueden fallar de maneras que ninguno exhibe por separado.

Por eso la unidad se estudia como una cadena de decisiones, cada una apoyada en la anterior: *medir* la línea de base de cada dependencia (percentiles, llamadas por pedido); *elegir el timeout* a partir de esa distribución y verificar que los plazos internos quepan en los externos; *decidir los reintentos* (cuáles fallas, cuántos, con qué espera y bajo qué presupuesto) sabiendo cuánto amplifican; *configurar el breaker* (qué cuenta como falla, política de apertura, tiempo abierto, pruebas en *half-open*); y *definir la degradación*, es decir, qué recibe el usuario cuando la dependencia no está. Si falta un eslabón, los demás no compensan: un *breaker* sin *timeout* no ve la falla lenta, y un *breaker* sin *fallback* falla rápido pero sigue fallando.

Errores de razonamiento frecuentes

- agregar reintentos sin evaluar cuánta carga suman cuando la causa es una saturación;
- reintentar en varias capas sin calcular el producto de intentos;
- elegir un *timeout* "a ojo" o desde el promedio, en vez de desde la distribución;
- medir la saturación con la CPU cuando el recurso que se agota son los hilos;
- configurar un *breaker* por fallas consecutivas sin considerar el volumen de tráfico;
- creer que un *breaker* sin *fallback* protege la experiencia del usuario, o que toda degradación es aceptable;
- configurar por separado mecanismos que interactúan (*load shedding* y balanceo de carga);
- considerar que más patrones aplicados significa más resiliencia.

Un sistema resiliente no es el que no falla, sino el que falla de la forma que se diseñó: rápido, acotado y visible. Los reintentos que en clases anteriores resolvían una pérdida de mensaje, frente a una dependencia saturada son el combustible de la cascada; y una dependencia lenta es más peligrosa que una caída.

14 Bibliografía guiada y ruta de profundización

Para profundizar se recomienda la siguiente ruta, con qué buscar en cada lectura:

- *Release It!*, cap. 2: seguir la cadena del caso de la aerolínea y marcar en qué eslabón actuaría cada mecanismo. Cap. 3: la terminología de falta, error y falla, y los puntos de corte. Cap. 4: puntos de integración, hilos bloqueados, reacciones en cadena y capacidades desbalanceadas; buscar qué recurso se agota en cada caso. Cap. 5: *timeouts*, *circuit breaker*, *bulkheads*, *fail fast*, *backpressure* y *load shedding*; buscar los supuestos de cada uno. Cap. 9, *Demand Control*: tiempo de residencia y dónde rechazar.

- *Site Reliability Engineering*, cap. 21: cómo decide Google si reintentar (presupuestos por pedido y por cliente). Cap. 22: la sección sobre reintentos y la de *load shedding*.
- *The Site Reliability Workbook*, cap. 11: reconstruir la secuencia de Pokémon GO y la de Dressy.
- Brooker (2015): las variantes de *jitter* y su comparación.

Durante la lectura, registre: modelo de sistema, propiedad perseguida, contraejemplo que motiva el mecanismo, costo principal y una condición bajo la cual no lo elegiría.

Referencias canónicas

1. **Nygard, M. T.** (2018). *Release It! Design and Deploy Production-Ready Software* (2.ª ed.). Pragmatic Bookshelf. (Capítulos 2, 3, 4, 5 y 9; en el 9, la sección *Demand Control*).
2. **Beyer, B., Jones, C., Petoff, J. y Murphy, N. R.** (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. (Capítulos 21, *Handling Overload*, y 22, *Addressing Cascading Failures*).
3. **Beyer, B., Murphy, N. R., Rensin, D. K., Kawahara, K. y Thorne, S.** (2018). *The Site Reliability Workbook: Practical Ways to Implement SRE*. O'Reilly Media. (Capítulo 11, *Managing Load*, de Bethea, Sheerin, Mace y King).
4. **Brooker, M.** (2015). *Exponential Backoff and Jitter*. AWS Architecture Blog.
5. **Little, J. D. C.** (1961). A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research*, 9(3), 383–387.

15 Aplicación al Trabajo Final Integrador

El grupo debe identificar cada dependencia remota de su sistema (servicios propios, bases de datos, APIs externas, colas) y decidir para cada una: el *timeout* (justificado con percentiles medidos y verificando que los plazos internos quepan en los externos), la política de reintentos (qué operaciones son idempotentes, cuántos reintentos, *backoff*, presupuesto), la configuración del *circuit breaker* (qué cuenta como falla, política de apertura, tiempo abierto) y la degradación aceptable, acordada con el dominio del problema. Los valores deben justificarse con datos; cuando todavía no se pueden medir, se registran como hipótesis con la medición que los confirmaría.

La decisión principal se documenta en un ADR breve, con: contexto (la dependencia y su rol), dos alternativas (por ejemplo, *breaker* por fallas consecutivas contra por tasa de fallas, o *fallback* contra falla explícita), supuesto decisivo (la tasa de error y el volumen esperados), garantía elegida, *trade-off* aceptado, evidencia pendiente y fecha de revisión. La evidencia mínima esperada es: percentiles de latencia por dependencia que justifiquen los *timeouts*, el cálculo de la amplificación por reintentos, y una prueba de falla (dependencia lenta o con errores) que muestre el comportamiento con y sin protección.